1.transform

Joao Lopes

2026-03-23

Contents

1. NUMERIC VECTORS **3**
1.1. COUNTS . . . 3
1.2. NUMERIC TRANSFORMATION . . . 4
Arithmetic and recycling rules . . . 4
Modular arithmetic . . . 4
Rounding . . . 4
Cutting numbers into ranges . . . 4
Cumulative and rolling aggregates . . . 5
1.3. GENERAL TRANSFORMATION . . . 5
Ranks . . . 5
Offsets . . . 6
Consecutive identifiers . . . 6

2. FACTORS **7**
2.1. BASICS . . . 7
2.2. DATASET gss_cat . . . 7
2.3. MODIFYING FACTOR ORDER . . . 7
2.4. MODIFYING FACTOR LEVELS . . . 9

3. LOGICAL VECTORS **11**
3.1. COMPARISONS . . . 11
Missing values . . . 11
3.2. BOOLEAN ALGEBRA . . . 11
Boolean operations . . . 11
Missing values . . . 11
Operator %in% . . . 12
3.3. SUMMARIES . . . 12
Logical summaries . . . 12
Numeric summaries of logical vectors . . . 12
Logical subsetting . . . 12
3.4. CONDITIONAL TRANSFORMATIONS . . . 13
Function if_else() . . . 13
Function case_when() . . . 13
Compatible types . . . 14

1. NUMERIC VECTORS

[from https://r4ds.hadley.nz/numbers.html]

```r
library("nycflights13") #collection of datasets
library("skimr")        #function skim() for descriptive statistics
library("tidyverse")    #collection of packages for data analysis
```

```r
#metadata
?flights
```

```r
#data inspection
glimpse(flights)

#descriptive statistics
skim(flights)
```

1.1. COUNTS

```r
#simple count
flights |> count(dest)

#sorted count
flights |> count(dest, sort = TRUE)

#count + summary statistics
flights |>
  group_by(dest) |>
  summarize(
    n = n(),
    delay = mean(arr_delay, na.rm = TRUE)
  )

#count distinct
flights |>
  group_by(dest) |>
  summarize(carriers = n_distinct(carrier)) |>
  arrange(desc(carriers))

#sum is weighted count
flights |> count(tailnum, wt = distance)

flights |>
  group_by(tailnum) |>
  summarize(miles = sum(distance))

#count missing values
flights |>
  group_by(dest) |>
  summarize(n_cancelled = sum(is.na(dep_time)))
```

1.2. NUMERIC TRANSFORMATION

Arithmetic and recycling rules

```
x <- c(1, 2, 10, 20)
x / 5
x / c(5, 5, 5, 5)
x * c(1, 2)
x * c(1, 2, 3)
```

Modular arithmetic

```
1:10 %/% 3
1:10 %% 3

flights |>
  mutate(
    hour = sched_dep_time %/% 100,
    minute = sched_dep_time %% 100,
    .keep = "used"
  )
```

Rounding

```
#simple round
round(123.456)

#round to nearest n digit
round(123.456, 2)  # two digits
round(123.456, 1)  # one digit
round(123.456, -1) # round to nearest ten
round(123.456, -2) # round to nearest hundred

#floor() vs ceiling()
round(c(1.5, 2.5))

x <- 123.456
floor(x)
ceiling(x)

#round down/up to nearest n digit
floor(x / 0.01) * 0.01
ceiling(x / 0.01) * 0.01

#round to nearest multiple
round(x / 4) * 4       # round to nearest multiple of 4
round(x / 0.25) * 0.25 # round to nearest 0.25
```

Cutting numbers into ranges

```
#simple cut
x <- c(1, 2, 6, 8, 10, 12)
cut(x, breaks = c(0, 5, 10, 15))
```

```r
cut(x,
    breaks = c(0, 5, 10, 15),
    labels = c("sm", "md", "lg")
)

#cut with values outside the range
y <- c(NA, -10, 6, 8, 200)
cut(y, breaks = c(0, 5, 10, 15))
```

Cumulative and rolling aggregates

```r
df <- tibble(
  x = 1:9,
  y = 9:1
)

#cumsum
df |> mutate(cumsum_x = cumsum(x), cumsum_y = cumsum(y))

#cumprod
df |> mutate(cumprod_x = cumprod(x), cumprod_y = cumprod(y))

#cummin
df |> mutate(cummin_x = cummin(x), cummin_y = cummin(y))

#cummax
df |> mutate(cummax_x = cummax(x), cummax_y = cummax(y))
```

1.3. GENERAL TRANSFORMATION

Ranks

```r
#simple ranks
x <- c(1, 2, 2, 3, 4, NA)
min_rank(x)
min_rank(desc(x))

#more ranks
df <- tibble(x = x)
df |>
  mutate(
    min_rank = min_rank(x),
    row_number = row_number(x),
    dense_rank = dense_rank(x),
    percent_rank = percent_rank(x),
    cume_dist = cume_dist(x)
  )

#using ranks to divide data
df <- tibble(id = 1:10)

df |>
  mutate(
```

```r
    row0 = row_number() - 1,
    three_groups = row0 %% 3
  )

df |>
  mutate(
    row0 = row_number() - 1,
    three_in_each_group = row0 %/% 3
  )
```

Offsets

```r
x <- c(2, 5, 11, 11, 19, 35)
lag(x)
lead(x)

x - lag(x)
x == lag(x)
```

Consecutive identifiers

```r
events <- tibble(
  time = c(0, 1, 2, 3, 5, 10, 12, 15, 17, 19, 20, 27, 28, 30)
)

#use lag() to detect large gaps
events <- events |>
  mutate(
    diff = time - lag(time, default = first(time)),
    has_gap = diff >= 5
  )
events

#use cumsum() to create periods of time
events |> mutate(
  group = cumsum(has_gap)
)
```

---

2. FACTORS

[from https://r4ds.hadley.nz/factors.html]

```
library("tidyverse") #collection of packages for data analysis
```

2.1. BASICS

```
#use just strings
x1 <- c("Dec", "Apr", "Jan", "Mar")
x2 <- c("Dec", "Apr", "Jam", "Mar")
sort(x1)

#simple factors
month_levels <- c(
  "Jan", "Feb", "Mar", "Apr", "May", "Jun",
  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
)
y1 <- factor(x1, levels = month_levels)
y1
sort(y1)

y2 <- factor(x2, levels = month_levels)
y2
```

```
#use fct()
y2 <- fct(x2, levels = month_levels)
```

2.2. DATASET gss_cat

```
#General Social Survey
gss_cat
```

```
?gss_cat
```

```
#look at factors
gss_cat |>
  count(marital)

gss_cat |>
  count(rincome)

gss_cat |>
  count(partyid)

gss_cat |>
  count(relig)
```

2.3. MODIFYING FACTOR ORDER

```
#fct_reorder() in ggplot
relig_summary <- gss_cat |>
  group_by(relig) |>
  summarize(
```

```r
    tvhours = mean(tvhours, na.rm = TRUE),
    n = n()
  )

ggplot(relig_summary, aes(x = tvhours, y = relig)) +
  geom_point()

ggplot(relig_summary, aes(x = tvhours, y = fct_reorder(relig, tvhours))) +
  geom_point()

#fct_reorder() in tibble
relig_summary |>
  mutate(
    relig = fct_reorder(relig, tvhours)
  ) |>
  ggplot(aes(x = tvhours, y = relig)) +
  geom_point()

#fct_relevel()
rincome_summary <- gss_cat |>
  group_by(rincome) |>
  summarize(
    age = mean(age, na.rm = TRUE),
    n = n()
  )

ggplot(rincome_summary, aes(x = age, y = fct_reorder(rincome, age))) +
  geom_point()

ggplot(rincome_summary, aes(x = age, y = fct_relevel(rincome, "Not applicable"))) +
  geom_point()

#fct_reorder2()
by_age <- gss_cat |>
  filter(!is.na(age)) |>
  count(age, marital) |>
  group_by(age) |>
  mutate(
    prop = n / sum(n)
  )

ggplot(by_age, aes(x = age, y = prop, color = marital)) +
  geom_line(linewidth = 1) +
  scale_color_brewer(palette = "Set1")

ggplot(by_age, aes(x = age, y = prop, color = fct_reorder2(marital, age, prop))) +
  geom_line(linewidth = 1) +
  scale_color_brewer(palette = "Set1") +
  labs(color = "marital")

#fct_infreq() and fct_rev()
gss_cat |>
  mutate(marital = marital |> fct_infreq() |> fct_rev()) |>
```

```r
  ggplot(aes(x = marital)) +
  geom_bar()
```

2.4. MODIFYING FACTOR LEVELS

```r
#ommit levels using fct_recode()
gss_cat |>
  mutate(
    partyid = fct_recode(partyid,
      "Republican, strong"    = "Strong republican",
      "Republican, weak"      = "Not str republican",
      "Independent, near rep" = "Ind,near rep",
      "Independent, near dem" = "Ind,near dem",
      "Democrat, weak"        = "Not str democrat",
      "Democrat, strong"      = "Strong democrat"
    )
  ) |>
  count(partyid)

#combine levels using fct_recode()
gss_cat |>
  mutate(
    partyid = fct_recode(partyid,
      "Republican, strong"    = "Strong republican",
      "Republican, weak"      = "Not str republican",
      "Independent, near rep" = "Ind,near rep",
      "Independent, near dem" = "Ind,near dem",
      "Democrat, weak"        = "Not str democrat",
      "Democrat, strong"      = "Strong democrat",
      "Other"                 = "No answer",
      "Other"                 = "Don't know",
      "Other"                 = "Other party"
    )
  )

#combine levels using fct_collapse()
gss_cat |>
  mutate(
    partyid = fct_collapse(partyid,
      "other" = c("No answer", "Don't know", "Other party"),
      "rep" = c("Strong republican", "Not str republican"),
      "ind" = c("Ind,near rep", "Independent", "Ind,near dem"),
      "dem" = c("Not str democrat", "Strong democrat")
    )
  ) |>
  count(partyid)

#lump unfrequent groups using fct_lump_lowfreq()
gss_cat |>
  mutate(relig = fct_lump_lowfreq(relig)) |>
  count(relig)

#lump unfrequent groups using fct_lump_n()
```

```r
gss_cat |>
  mutate(relig = fct_lump_n(relig, n = 10)) |>
  count(relig, sort = TRUE)
```

---

3. LOGICAL VECTORS

[from https://r4ds.hadley.nz/logicals.html]

```
library("nycflights13") #collection of datasets
library("tidyverse")    #collection of packages for data analysis
```

3.1. COMPARISONS

```
#create logical vector inline
flights |>
  filter(dep_time > 600 & dep_time < 2000 & abs(arr_delay) < 20)

#create logical vector outside logical condition
flights |>
  mutate(
    daytime = dep_time > 600 & dep_time < 2000,
    approx_ontime = abs(arr_delay) < 20,
  ) |>
  filter(daytime & approx_ontime)
```

Missing values

```
#logical conditions
NA > 5
10 == NA
NA == NA
flights |>
  filter(dep_time == NA)

#is.na()
flights |>
  filter(is.na(dep_time))
```

3.2. BOOLEAN ALGEBRA

Boolean operations

```
x <- c(rep(TRUE,6),rep(FALSE,3))
y <- c(rep(FALSE,3),rep(TRUE,6))
tibble(id = c(1:9), x, y)

which(x)          #x
which(y)          #y
which(x & !y)     #x & !y
which(!x & y)     #!x & y
which(xor(x, y)) #xor(x, y)
which(x | y)      # x | y
```

Missing values

```
df <- tibble(x = c(TRUE, FALSE, NA), `NA` = NA)
```

```r
df |>
  mutate(
    x_and_NA = x & `NA`,
    x_or_NA = x | `NA`
  )
```

Operator %in%

```r
flights |>
  filter(month == 11 | month == 12)

flights |>
  filter(month %in% c(11, 12))

c(1, 2, NA) == 1
c(1, 2, NA) %in% 1
```

3.3. SUMMARIES

Logical summaries

```r
flights |>
  group_by(year, month, day) |>
  summarize(
    all_delayed = all(dep_delay <= 60, na.rm = TRUE),
    any_long_delay = any(arr_delay >= 300, na.rm = TRUE),
    .groups = "drop"
  )
```

Numeric summaries of logical vectors

```r
flights |>
  group_by(year, month, day) |>
  summarize(
    proportion_delayed = mean(dep_delay <= 60, na.rm = TRUE),
    count_long_delay = sum(arr_delay >= 300, na.rm = TRUE),
    .groups = "drop"
  )
```

Logical subsetting

```r
flights |>
  filter(arr_delay > 0) |>
  group_by(year, month, day) |>
  summarize(
    behind = mean(arr_delay),
    n = n(),
    .groups = "drop"
  )
flights |>
  filter(arr_delay < 0) |>
  group_by(year, month, day) |>
```

```r
  summarize(
    behind = mean(arr_delay),
    n = n(),
    .groups = "drop"
  )

flights |>
  group_by(year, month, day) |>
  summarize(
    behind = mean(arr_delay[arr_delay > 0], na.rm = TRUE),
    ahead = mean(arr_delay[arr_delay < 0], na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

3.4. CONDITIONAL TRANSFORMATIONS

Function if_else()

```r
#simple if_else()
x <- c(-3:3, NA)
if_else(x > 0, "is_pos", "is_neg")
if_else(x > 0, "is_pos", "is_neg", "is_???")
if_else(x < 0, -x, x)

#sequence of if_else()
if_else(x == 0, "0",
if_else(x < 0, "is_neg",
               "is_pos"),
               "is_???")
```

Function case_when()

```r
case_when(
  x == 0   ~ "0",
  x < 0    ~ "is_neg",
  x > 0    ~ "is_pos",
  is.na(x) ~ "is_???"
)

case_when(
  x < 0 ~ "is_neg",
  x > 0 ~ "is_pos",
)

case_when(
  x < 0 ~ "is_neg",
  x > 0 ~ "is_pos",
  TRUE  ~ "is_???"
)

case_when(
  x > 0 ~ "is_pos",
```

```r
  x > 2 ~ "is_big"
)

flights |>
  mutate(
    status = case_when(
      is.na(arr_delay)      ~ "cancelled",
      arr_delay < -30       ~ "very early",
      arr_delay < -15       ~ "early",
      abs(arr_delay) <= 15  ~ "on time",
      arr_delay < 60        ~ "late",
      arr_delay < Inf       ~ "very late",
    ),
    .keep = "used"
  )
```

Compatible types

```r
if_else(TRUE, "a", 1)

case_when(
  x < -1 ~ TRUE,
  x > 0  ~ now()
)
```